

Exercícios – aula 5

Luís Felipe

UFF

10 de Setembro de 2025

Encontrar o maior número

Problema: Ler n números e imprimir o maior.

Ideia:

- Ler o primeiro número e armazenar em maior.
- Comparar os próximos e atualizar se encontrar um maior.

Implementação do maior número

```
1 n = int(input("Digite um número: "))
2 maior = int(input())
3 cont = 1
4
5 while cont < n:
6     aux = int(input())
7     if aux > maior:
8         maior = aux
9     cont += 1
10
11 print("O maior número é:", maior)
```

Números de Fibonacci

A **série de Fibonacci** é uma sequência onde cada termo é a soma dos dois anteriores:

$$F(n) = F(n - 1) + F(n - 2), \quad \text{com } F(1) = 1 \text{ e } F(2) = 1$$

Primeiros elementos da sequência:

1, 1, 2, 3, 5, 8, 13, 21, ...

Problema: Fazer um programa que imprime os primeiros n números da série de Fibonacci.

Lógica:

- Começamos com $f_ant = 0$ e $f_atual = 1$.
- A cada passo, imprimimos f_atual , atualizamos os valores e repetimos até imprimir n termos.

Fibonacci com while

```
1 n = int(input("Digite um número: "))
2 cont = 0
3 f_ant = 0
4 f_atual = 1
5
6 while cont < n:
7     print(f_atual, ", ", ", end="") # imprime o número
8     cont = cont + 1                # conta mais um
9     f_aux = f_atual                # guarda o valor atual
10    f_atual = f_atual + f_ant       # atualiza f_atual
11    f_ant = f_aux                  # atualiza f_ant
```

Saída esperada para $n = 6$:

```
1 1, 1, 2, 3, 5, 8,
```

Laços Encaixados

- Um laço (**for** ou **while**) pode conter outro laço em seu bloco.
- Essa estrutura é útil para trabalhar com dados organizados em pares, matrizes, tabelas, etc.
- Cada vez que o laço externo dá um passo, o laço interno executa completamente.

Exemplo:

```
1 for i in range(1, 11):  
2     for j in range(1, 6):  
3         print(i, j)
```

Saída (parcial):

```
1 1 1  
2 1 2  
3 1 3  
4 1 4  
5 1 5  
6 2 1  
7 ...  
8 10 5
```

Entendendo os Laços Encaixados

- O laço externo (i) controla as repetições maiores.
- O laço interno (j) reinicia a cada iteração do externo.
- No exemplo: para cada i de 1 a 10, j vai de 1 a 5.

Total de linhas impressas: $10 \times 5 = 50$

Esse padrão é comum em algoritmos que processam linhas e colunas, ou combinações de pares de valores.

Tabuada com Laços Encaixados

Exercício: Implemente a tabuada de 1 até 5, indicando qual tabuado será apresentado antes de exibi-la.

```
1 # Tabuada de 1 a 5
2 for i in range(1, 6):
3     print(f"Tabuada do {i}:")
4     for j in range(1, 11):
5         print(f"{i} x {j} = {i*j}")
6     print()                                # linha em branco entre tabuadas
```

Saída (parcial):

```
1 Tabuada do 1:
2 1 x 1 = 1
3 1 x 2 = 2
4 ...
5 1 x 10 = 10
6
7 Tabuada do 2:
8 2 x 1 = 2
9 ...
10 2 x 2 = 4
11 .
12 .
13 .
```

Laços Encaixados: Todos os Resultados com 4 Dados

- **Problema:** imprimir todas as combinações possíveis ao jogar 4 dados de 6 faces.
- Solução: usar 4 laços for aninhados, um para cada dado.
- Cada laço representa um dado e vai de 1 a 6.

Código em Python:

```
1 print("D1 D2 D3 D4")
2 for d1 in range(1, 7):
3     for d2 in range(1, 7):
4         for d3 in range(1, 7):
5             for d4 in range(1, 7):
6                 print(d1, d2, d3, d4)
```

Total de combinações: $6^4 = 1296$

Abreviações

Exercício: Leia um número inteiro N , representando a quantidade de nomes completos que serão digitados. Em seguida, leia N nomes completos (cada um com pelo menos um primeiro nome e um sobrenome).

O programa deve imprimir cada nome abreviado no seguinte formato: Sobrenome, iniciais dos demais nomes em ordem, todas com letra maiúscula e seguidas de ponto.

Exemplo:

Entrada: Luís Felipe Ignácio Cunha

Saída: Cunha, L. F. I.

Atividade para dia 15/09.

Implementação dos nomes abreviados

```
1 # Lê a quantidade de nomes
2 n = int(input())
3
4 for k in range(n):
5     nome_completo = input()
6     partes = nome_completo.split()
7
8     # o sobrenome é a última palavra
9     sobrenome = partes[len(partes)-1]
10
11     # pegamos as iniciais dos outros nomes
12     iniciais = []
13     for i in range(len(partes)-1):
14         palavra = partes[i]
15         for letra in palavra:
16             inicial = letra.upper() + "."
17             break # pega só a primeira letra
18         iniciais.append(inicial)
19
20     # imprimindo no formato desejado
21     print(sobrenome + ", ", end=" ")
22     for inicial in iniciais:
23         print(inicial, end=" ")
24     print()
```

Adaptação

Altere o código do exercício anterior para que ele não peça primeiro a quantidade de nomes.

Em vez disso, o programa deve ler nomes completos um a um, gerar suas abreviações, e continuar esse processo indefinidamente até que seja digitada uma linha em branco (sem nome), momento em que o programa deve parar.

Separe com uma quebra de linha cada resultado da próxima entrada.

Implementação dos nomes abreviados adaptado

```
1 nome = input()
2
3 while nome != "":
4     partes = nome.split()
5
6     # o sobrenome é a última palavra
7     sobrenome = partes[len(partes)-1]
8
9     # pegamos as iniciais dos outros nomes
10    iniciais = []
11    for i in range(len(partes)-1):
12        palavra = partes[i]
13        for letra in palavra:
14            inicial = letra.upper() + "."
15            break # pega só a primeira letra
16        iniciais.append(inicial)
17
18    # imprimindo no formato desejado
19    print(sobrenome + ", ", end=" ")
20    for inicial in iniciais:
21        print(inicial, end=" ")
22    print("\n")
23    nome = input()
```